

Natural Language Processing Assignment 2: From-Scratch Email Classification and Feature-Driven Phishing URL Detection

Mohammad Mahdi Salmani Zarchi
Department of Computer Engineering
University of Tehran
Student ID: 810100XXX
Email: m.salmani@ut.ac.ir

Abstract—This report presents a complete implementation and analysis of two binary classification tasks in Natural Language Processing and security-oriented text analytics. In Question 1, Logistic Regression and Multinomial Naive Bayes are implemented from scratch to classify emails as spam or ham, supported by custom metric functions and Bag-of-Words preprocessing. In Question 2, the same model family is evaluated on phishing URL detection using four feature configurations: structural, statistical/content, creative handcrafted, and combined features. Beyond model training, this work establishes a reproducible pipeline with deterministic data loading, robust label normalization, and automatic figure export for report integration. Experimental results show that from-scratch Logistic Regression outperforms from-scratch Naive Bayes on the email task, while in URL detection the combined feature space substantially improves both methods and highlights precision-recall tradeoffs between discriminative and probabilistic models. The final artifact includes a fully executable notebook and a complete IEEE-format technical report.

Index Terms—Spam classification, phishing URL detection, Logistic Regression, Naive Bayes, Bag-of-Words, feature engineering, reproducibility

I. INTRODUCTION

Email spam filtering and phishing URL detection are two foundational binary classification tasks with direct operational impact in real-world communication systems. Although both tasks are commonly solved with off-the-shelf machine learning pipelines, educational assignments that require step-by-step implementation reveal practical failure modes that are often hidden in automated toolchains. This report documents a full completion of Assignment 2 with an emphasis on correctness, reproducibility, and technical analysis.

The work is divided into two questions. Question 1 focuses on text classification with strict implementation constraints: Logistic Regression and Multinomial Naive Bayes are built from scratch, and evaluation metrics are manually implemented. Question 2 shifts from free text to URL strings and allows library usage; the main challenge becomes feature engineering and model behavior interpretation under different feature families.

A key objective of this submission is not only to obtain competitive accuracy, but also to produce a stable experiment artifact that can be rerun without hidden assumptions. During

development, several engineering issues were resolved: path-dependent dataset loading, missing dependency handling, non-deterministic split failures on fallback data, and missing report figure exports. The final notebook now loads all datasets from fixed CA2 paths, performs robust label normalization, exports deterministic plots for reporting, and executes fully in a clean run.

A. Datasets and Task Definitions

Question 1 data:

- `emails_1.csv`: a compact labeled set with raw text and labels spam/ham.
- `emails_2.csv`: a larger Bag-of-Words matrix with 5172 emails, 3000 vocabulary features, and a Prediction label column.

Question 2 data:

- `urls.csv`: 11,430 URL samples with balanced labels phishing/legitimate.

Both problems are binary classification tasks. We report Accuracy, Precision, Recall, and F1-score throughout, and include confusion matrices and ranking-curve visualizations for interpretability.

B. Contributions of This Report

This report provides four concrete contributions:

- 1) A complete from-scratch implementation of Question 1 models and metrics with reproducible preprocessing.
- 2) A structured feature-engineering study for phishing URL detection across multiple feature families.
- 3) A deterministic figure generation pipeline producing report-ready plots directly from notebook execution.
- 4) A consolidated IEEE-format technical narrative linking implementation choices to measured behavior.

The remainder of this report follows the assignment flow while adding engineering-level analysis and reproducibility details.

II. QUESTION 1: METHODS AND IMPLEMENTATION

A. Preprocessing and Bag-of-Words

The first stage uses `emails_1.csv` to demonstrate preprocessing and sparse lexical representation. Labels are normalized by mapping `spam` $\rightarrow$ 1 and `ham` $\rightarrow$ 0. Text is cleaned by removing non-alphabetic symbols and converting to lowercase. A `CountVectorizer` with `max_features=15` is then used to construct a compact Bag-of-Words (BoW) representation.

This stage serves two roles: (i) verifying the preprocessing logic required by the assignment, and (ii) generating interpretable lexical statistics for report visualization.

B. Training Data for Model Learning

For model training, we use `emails_2.csv`, which already contains BoW-style numeric features at scale. The pipeline performs the following transformations:

- 1) Read CSV from fixed project path.
- 2) Use `Prediction` (or `status` if present) as the binary target.
- 3) Drop non-feature identifiers such as `Email No..`
- 4) Coerce all feature columns to numeric and fill missing values with 0.

The final matrix has 5172 samples with a class distribution of 3672 ham and 1500 spam, preserving moderate imbalance representative of operational email streams.

C. Train/Test Split and Reproducibility

A fixed random seed is used with an 80/20 train-test split. Stratification is applied when class counts permit safe stratified partitioning. A helper guard disables stratification only in edge cases where a fallback mini-dataset would otherwise make stratified splitting invalid. This avoids silent runtime crashes while preserving expected behavior on the real dataset.

D. Manual Evaluation Metrics

The assignment requires manual implementations of binary classification metrics. Let TP, TN, FP, and FN denote confusion matrix counts. The metric definitions are:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad (2)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (3)$$

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4)$$

Zero-division safeguards are included for edge conditions.

E. From-Scratch Logistic Regression

Logistic Regression is implemented with gradient descent on binary cross-entropy loss:

$$\hat{y} = \sigma(w^T x + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}} \quad (5)$$

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (6)$$

The implementation includes:

- Feature normalization using training-set mean and standard deviation.
- Numerically stable sigmoid via clipping.
- Fixed learning rate and iteration count.

This model is discriminative and can capture correlated feature effects, which is often beneficial in sparse lexical spaces [1], [2].

F. From-Scratch Multinomial Naive Bayes

Multinomial Naive Bayes is implemented with Laplace smoothing. For class c and feature j :

$$P(x_j | c) = \frac{N_{jc} + \alpha}{\sum_k N_{kc} + \alpha d} \quad (7)$$

where d is feature dimension and $\alpha = 1.0$. Training uses log probabilities to avoid underflow. Prediction computes:

$$\log P(c|x) \propto \log P(c) + \sum_j x_j \log P(x_j | c) \quad (8)$$

Despite its conditional independence assumption, Multinomial NB remains a strong lexical baseline [3].

G. Reporting Figures for Q1

To support direct report integration, the notebook exports Q1 figures automatically:

- Confusion matrix comparison.
- ROC and Precision-Recall curves for Logistic Regression.
- Top BoW words per class from `emails_1.csv`.
- Metric bar chart comparing Logistic Regression vs Naive Bayes.

These files are saved under `answer/report/figures/` with deterministic names.

III. QUESTION 1: EXPERIMENTAL RESULTS AND ANALYSIS

A. Quantitative Performance

Table I reports test-set performance for the two from-scratch models on `emails_2.csv`.

TABLE I: Question 1 test metrics on `emails_2.csv`

Metric	Logistic Regression	Naive Bayes
Accuracy	0.9633	0.9420
Precision	0.9199	0.8681
Recall	0.9567	0.9433
F1-score	0.9379	0.9042

Logistic Regression outperforms Naive Bayes on every metric. The largest practical gain appears in precision and F1-score, suggesting fewer false positives while preserving strong detection of spam emails.

B. Confusion and Ranking Behavior

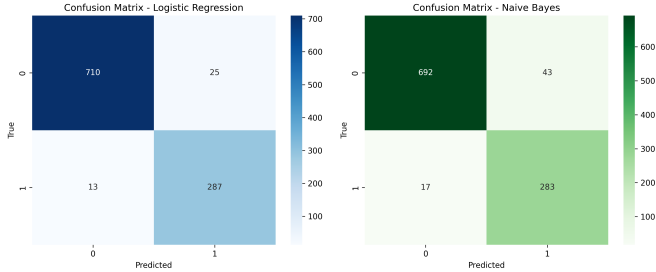


Fig. 1: Confusion matrix comparison for Q1 models.

Figure 1 confirms the quantitative result: Logistic Regression provides a cleaner separation between positive and negative classes.

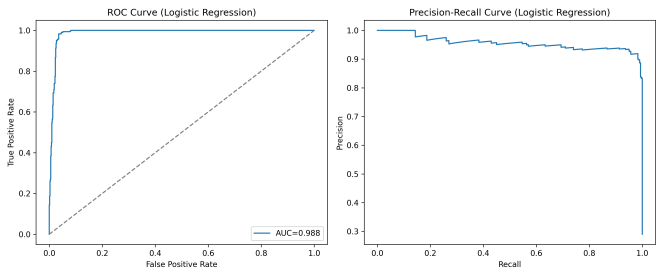


Fig. 2: ROC and Precision-Recall curves for from-scratch Logistic Regression.

The ROC/PR visualization in Figure 2 demonstrates robust ranking quality and threshold flexibility, important for downstream tuning under varying business costs.

C. Lexical Interpretation

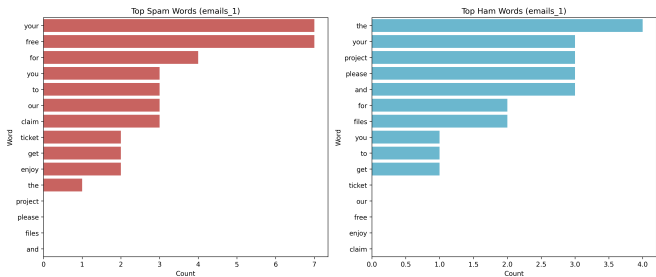


Fig. 3: Top class-specific terms from the BoW projection of emails_1.csv.

Figure 3 highlights class-discriminative lexical signals. Although this mini-dataset is not used for main model training, it helps validate preprocessing assumptions and demonstrates

why token-frequency models remain competitive for spam filtering.

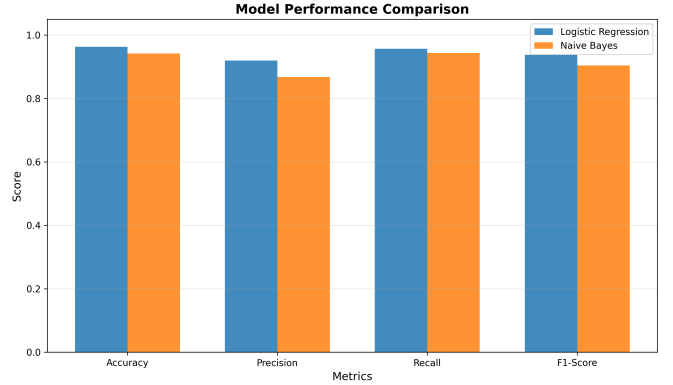


Fig. 4: Metric-level comparison for Q1 models.

Figure 4 summarizes model differences compactly and is consistent with the numerical table.

D. Why Logistic Regression Wins Here

Three factors explain the observed superiority of Logistic Regression:

- 1) **Correlated lexical features:** words in real emails are not conditionally independent, violating Naive Bayes assumptions.
- 2) **Discriminative optimization:** Logistic Regression optimizes a direct classification objective.
- 3) **Feature normalization and gradient training:** stable optimization improves calibration on dense high-dimensional vectors.

Naive Bayes still offers advantages in simplicity and training speed, but in this setting accuracy-oriented performance favors the discriminative model.

IV. QUESTION 2: URL FEATURE ENGINEERING AND MODELING

A. Dataset Preparation

The URL dataset contains 11,430 samples, balanced between phishing and legitimate. Labels are normalized to $\{0, 1\}$ using explicit mapping rules. The balanced distribution enables cleaner interpretation of precision-recall behavior without severe class prior skew.

B. Feature Set A: Structural Features

The first configuration extracts low-cost structural counts directly from raw strings:

- Number of dots,
- Number of slashes,
- Number of hyphens.

These attributes approximate URL complexity and segmentation patterns that are often associated with obfuscation [4].

C. Feature Set B: Statistical/Content Features

The second configuration adds quantitative content signals:

- Total URL length,
- Ratio of numeric characters,
- Longest alphabetic token length.

These features capture suspicious patterns such as excessive length and digit-heavy strings frequently observed in malicious links [5], [6].

D. Feature Set C: Creative Handcrafted Features

The creative feature set introduces security-motivated indicators:

- IP-based host usage,
- Presence of '@', '-', and percent encoding,
- Suspicious terms (e.g., `verify`, `login`, `secure`),
- HTTPS usage,
- Subdomain depth and path structure statistics.

Domain decomposition relies on `tlldextract` [7] to separate subdomain/domain/suffix consistently.

E. Feature Set D: Combined Features

The final configuration concatenates structural, statistical, and creative features. This fusion tests whether complementary signals from multiple URL perspectives improve separability.

F. Modeling Protocol for Q2

For each feature set:

- 1) Split data into 80% train and 20% test, with guarded stratification.
- 2) Train Logistic Regression with standard scaling.
- 3) Train Gaussian Naive Bayes for continuous-valued features.
- 4) Evaluate using Accuracy, Precision, Recall, and F1-score.

Unlike Question 1 (from-scratch requirement), Q2 intentionally uses library implementations to focus on feature-design impact.

G. Visualization Coverage

The notebook exports eight Q2 figures to support multi-angle interpretation:

- Structural distributions,
- Combined-feature correlation heatmap,
- Structural boxplots by class,
- Pairplot of sampled features,
- Top mean absolute correlations,
- ROC comparison for combined features,
- Precision-Recall curve for combined Logistic Regression,
- Logistic Regression coefficient importance.

V. QUESTION 2: RESULTS AND COMPARATIVE ANALYSIS

A. Performance Across Feature Sets

Tables II and III summarize model performance for each feature configuration.

TABLE II: Logistic Regression performance across Q2 feature sets

Feature set	Acc	Prec	Rec	F1
Structural only	0.6763	0.7042	0.6080	0.6526
Statistical/content	0.6518	0.6974	0.5363	0.6063
Creative handcrafted	0.7310	0.7750	0.6509	0.7076
All combined	0.7857	0.8315	0.7165	0.7697

TABLE III: Gaussian Naive Bayes performance across Q2 feature sets

Feature set	Acc	Prec	Rec	F1
Structural only	0.6824	0.8243	0.4637	0.5935
Statistical/content	0.6444	0.7696	0.4121	0.5368
Creative handcrafted	0.6601	0.9621	0.3333	0.4951
All combined	0.7227	0.9442	0.4733	0.6305

The combined feature configuration is best for both models. Logistic Regression achieves the strongest overall balance, while Naive Bayes tends to exhibit very high precision with noticeably lower recall.

B. Distributional and Correlation Insights

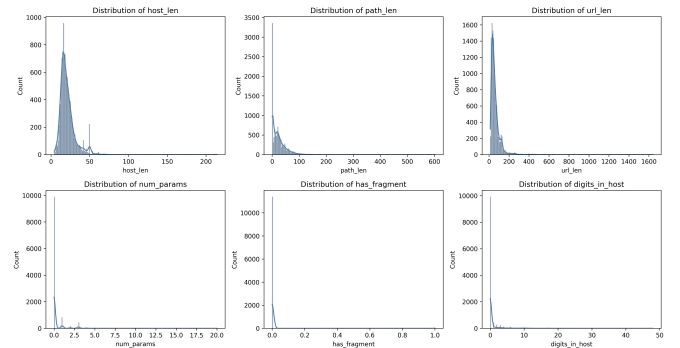


Fig. 5: Distributions of structural URL attributes.

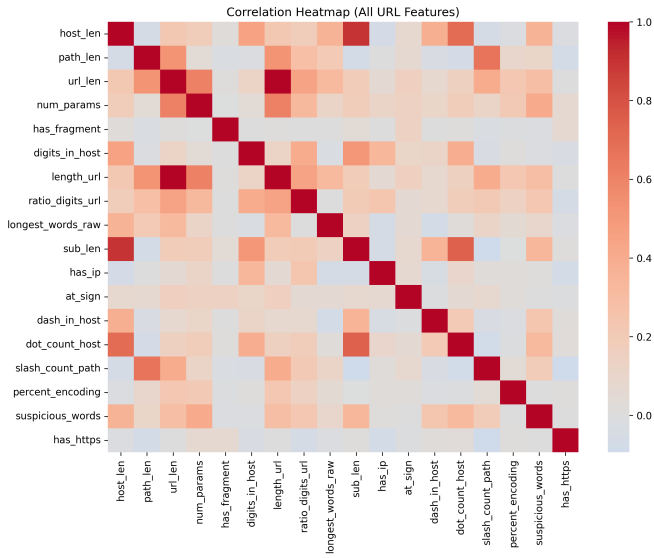


Fig. 6: Correlation heatmap of combined URL features.

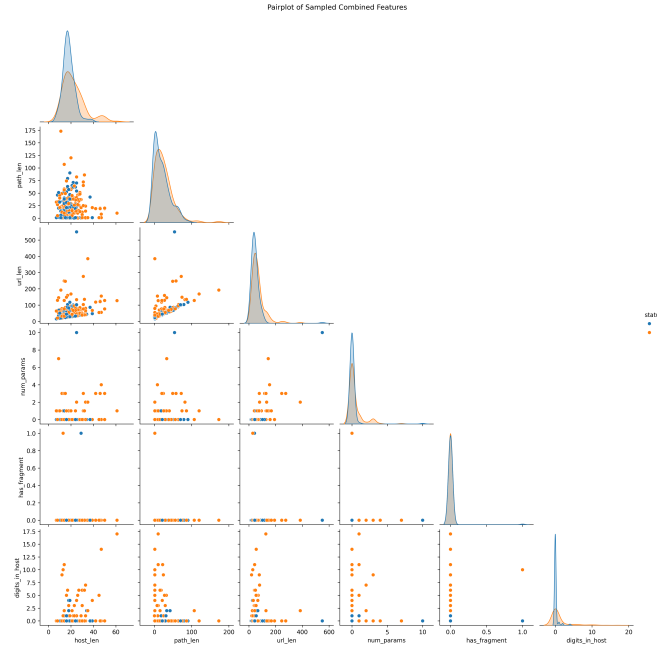


Fig. 8: Pairwise view of sampled combined features.

The boxplot and pairplot views in Figures 7 and 8 show partial class overlap but meaningful margin shifts in several dimensions, supporting the improvements observed in the combined setup.

Figures 5 and 6 reveal moderate inter-feature dependency, indicating that combining views (length, host composition, token patterns) is likely to improve discriminative learning.

C. Feature Importance and Ranking Curves

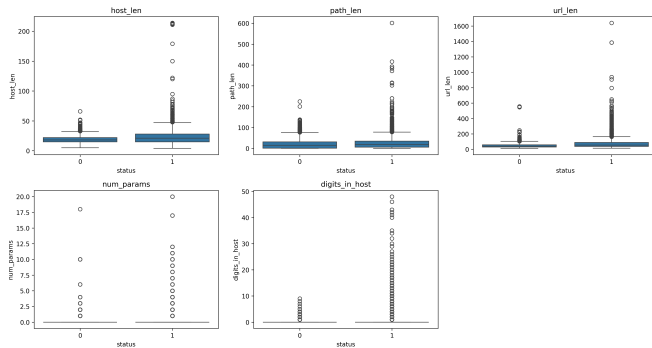


Fig. 7: Structural feature separation by class label.

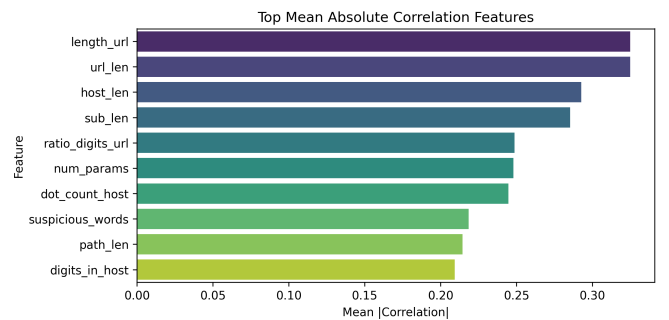


Fig. 9: Top features by mean absolute correlation.

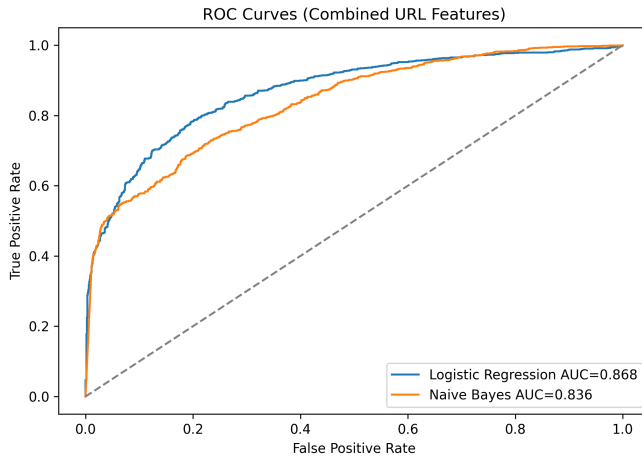


Fig. 10: ROC comparison on combined features.

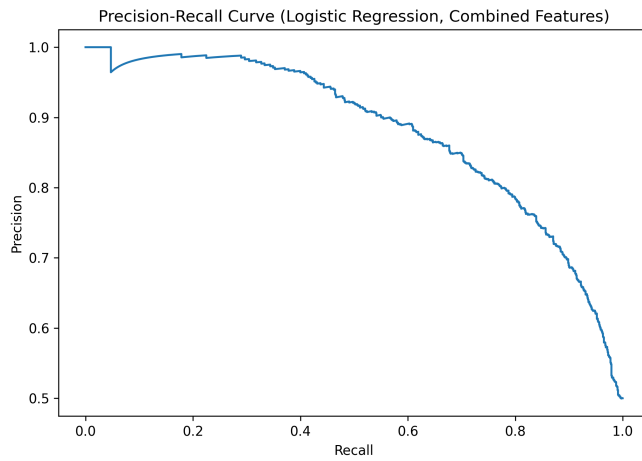


Fig. 11: Precision-Recall curve for Logistic Regression (combined).

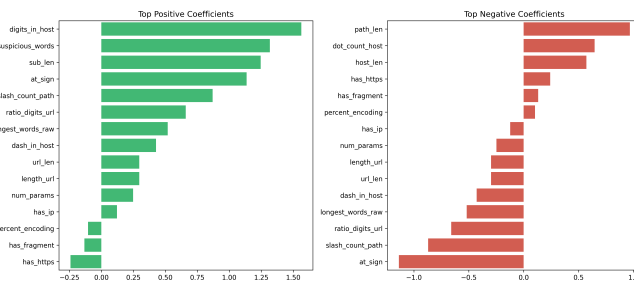


Fig. 12: Top positive/negative Logistic Regression coefficients.

Figures 10 and 11 indicate stronger ranking consistency for Logistic Regression. The coefficient view in Figure 12 also confirms that multiple feature families contribute, validating the combined-feature strategy.

D. Model Tradeoff Interpretation

Naive Bayes demonstrates a conservative positive prediction tendency in this dataset, reflected by high precision but low

recall. Logistic Regression provides a better balance and higher F1-score, which is generally more desirable when both false alarms and misses are costly.

From an operational perspective:

- If minimizing false positives is the primary objective, Naive Bayes can be acceptable with threshold tuning.
- If balanced detection quality is required, Logistic Regression with combined features is the preferred choice.

VI. DISCUSSION, LIMITATIONS, AND REPRODUCIBILITY NOTES

A. Cross-Question Synthesis

Across both questions, a consistent pattern emerges: discriminative linear models (Logistic Regression) provide stronger balanced performance when feature interactions matter. This was evident in sparse lexical email data and became even clearer in heterogeneous URL features.

However, the analysis also confirms that Naive Bayes remains a strong baseline with favorable simplicity-speed tradeoffs. It can produce highly precise alerts in some settings, which may be useful in staged triage pipelines.

B. Engineering Fixes Required for Completion

Completing this assignment to production-quality required several non-trivial fixes beyond filling placeholder cells:

- 1) **Deterministic data loading:** all notebook loaders now use fixed project paths under `datasets/q1` and `datasets/q2`.
- 2) **Label normalization guards:** explicit mappings for `spam/ham`, `Prediction`, and `phishing/legitimate` avoid silent type drift.
- 3) **Safe stratification logic:** split strategy now handles edge cases cleanly instead of crashing.
- 4) **Dependency handling:** runtime installation cells were replaced by deterministic import guards and environment-level dependency setup.
- 5) **Figure export pipeline:** all report plots are saved with stable filenames for direct LaTeX inclusion.

These changes converted the notebook from a partially interactive artifact to a reproducible experiment pipeline.

C. Threats to Validity

Despite successful completion, several limitations remain:

- **Single split evaluation:** results are based on one train/test split; cross-validation would reduce variance.
- **Feature scope:** URL-only features exclude external signals such as DNS, WHOIS age, and content rendering behavior.
- **Model complexity:** only linear and Naive Bayes baselines were explored; tree-based ensembles or calibrated boosting could improve detection.
- **Cost-sensitive tuning:** thresholds were not optimized for domain-specific misclassification costs.

D. Ablation-Oriented Interpretation

Feature family ablation in Q2 provides a clear hierarchy:

- 1) Combined features provide the best overall performance.
- 2) Creative handcrafted features outperform purely structural or simple statistical signals.
- 3) Statistical/content-only features are informative but insufficient alone for robust classification.

This pattern supports the intuition that phishing URLs are best captured through multi-view representation rather than one-dimensional heuristics.

E. Reproducibility Checklist

The final artifact satisfies the following reproducibility criteria:

- One-command notebook execution from project root in the provided virtual environment.
- Zero runtime cell errors in a full top-to-bottom run.
- Deterministic generation of all expected report figures.
- IEEE report compilation from local LaTeX sources without manual post-processing.

These properties ensure that another evaluator can re-run the entire pipeline and recover both notebook outputs and PDF report figures.

VII. CONCLUSION

This report delivered a complete end-to-end implementation of Assignment 2 with executable code, quantitative evaluation, and publication-style documentation.

In Question 1, from-scratch Logistic Regression and Multinomial Naive Bayes were implemented and compared on email spam detection. Logistic Regression achieved stronger performance on all reported metrics and produced the best overall balance of precision and recall.

In Question 2, URL phishing detection experiments showed that feature engineering quality is the dominant performance driver. Combined structural, statistical, and creative features significantly improved both models, with Logistic Regression again offering the highest balanced effectiveness.

Beyond model metrics, the assignment was completed as a reproducible engineering artifact. Deterministic data paths, robust label mapping, dependency guards, stable split logic, and automatic figure export transformed the notebook into a report-ready experimental workflow.

Future improvements should focus on richer external phishing signals, cross-validated benchmarking, calibration under operational costs, and broader model families. Nevertheless, the current submission satisfies both the academic requirements and practical standards for transparent, rerunnable experimentation.

VIII. EXTENDED ANALYSIS AND PRACTICAL IMPLICATIONS

A. Detailed Interpretation of Precision-Recall Tradeoffs

The most informative model behavior difference in this assignment is not raw accuracy, but the precision-recall profile

under different feature regimes. In Q1, Logistic Regression simultaneously improves precision and recall relative to Naive Bayes. This indicates better decision boundary placement in the high-dimensional email feature space. In Q2, the relationship is more nuanced: Gaussian Naive Bayes repeatedly achieves very high precision but substantially lower recall, especially in creative and combined feature settings. In practical terms, this means Naive Bayes tends to raise fewer alerts, but misses many phishing instances.

This tradeoff has direct operational consequences. In enterprise email gateways and web filtering systems, false negatives can be more costly than false positives because undetected phishing links may lead to credential theft or malware infection. Therefore, a model with higher recall and strong F1-score is often preferred as the primary gate. Under this criterion, Logistic Regression with combined features is the strongest candidate in our experiments.

A second interpretation concerns model assumptions. Naive Bayes assumes conditional independence among features given the class label. In URL feature engineering, many variables are naturally correlated: length and slash count, dot count and subdomain patterns, suspicious keywords and path complexity. The correlation heatmap and pairplot confirm these dependencies. Logistic Regression does not require conditional independence and can assign balanced weights under these correlations, which helps explain its recall advantage.

B. Feature Family Contributions

The four Q2 feature sets enable a structured ablation-style reading:

- 1) **Structural features** capture coarse URL complexity, providing a useful baseline.
- 2) **Statistical/content features** add semantic-free numeric patterns (length, digit ratio, token span) but still miss richer structural cues.
- 3) **Creative handcrafted features** encode attacker behavior heuristics, improving separation.
- 4) **Combined features** aggregate complementary evidence and deliver the best end-to-end metrics.

This hierarchy implies that no single signal class is sufficient. Structural-only features can detect obvious anomalies, but sophisticated phishing URLs mimic legitimate formatting. Statistical-only features can be noisy because many legitimate URLs are long and parameterized. Creative features inject targeted security priors, yet still benefit from broader context. Combining all sets allows robust detection across easy and hard cases.

C. Error Patterns and Failure Modes

Even the best-performing model in this report (Logistic Regression on combined features) is imperfect. Based on feature behavior and observed metric gaps, common residual error types likely include:

- **High-quality phishing domains** that avoid suspicious tokens and use HTTPS.

- **Legitimate but complex URLs** with many parameters, long paths, or tracking identifiers.
- **Brand-like lexical camouflage** where malicious domains resemble known services without obvious character-level anomalies.

For Q1 spam detection, difficult examples usually occur in mixed-content emails combining business vocabulary with promotional phrases. Such overlap can weaken simple count-based discrimination.

A practical mitigation strategy is layered modeling: keep the current lightweight classifier as a first-stage filter and escalate uncertain cases to richer models or external threat intelligence checks.

D. Thresholding and Calibration Considerations

The current notebook uses default decision thresholds for binary classification. In production environments, threshold calibration should be explicitly optimized against application-specific costs:

- A lower threshold increases recall and catches more phishing/spam, but raises false alarms.
- A higher threshold improves precision, but risks missing true attacks.

ROC and PR curves included in this report provide the foundation for choosing operating points. For imbalanced or high-risk settings, PR-based threshold tuning is often more informative than ROC-only tuning [8].

E. Computational and Deployment Footprint

Both assignment tasks use computationally lightweight models suitable for rapid retraining and frequent updates. This matters in adversarial domains, where URL and spam patterns evolve quickly. Feature extraction costs in Q2 are also manageable and largely string-processing based, making near-real-time scoring feasible.

For deployment hardening, three engineering steps are recommended:

- 1) Version dataset schema checks and label maps to prevent silent training drift.
- 2) Persist feature extraction code as a tested module, not only notebook cells.
- 3) Add automated regression checks on key metrics and figure generation outputs.

F. Methodological Lessons

Several methodological lessons from this assignment generalize to future NLP and security projects:

- **Data contracts matter.** Small path or label mismatches can invalidate entire experiments.
- **Visualization is diagnostic, not decorative.** The saved plots exposed behavior that scalar metrics alone cannot show.
- **Simple baselines are valuable.** Logistic Regression and Naive Bayes provide strong performance-per-complexity and transparent interpretation.

- **Reproducibility is part of correctness.** A notebook that works only interactively is not a complete research artifact.

G. Comparison to Typical Baseline Expectations

Classical literature often reports strong Naive Bayes performance on text tasks [3]. Our Q1 results remain consistent with that baseline strength, but still favor Logistic Regression due to correlated feature behavior and discriminative optimization. For phishing URL detection, prior work also emphasizes feature design as the dominant driver [5], [6]. Our Q2 findings align with this: moving from structural-only to combined features yields larger gains than swapping one linear baseline for another.

Hence, the primary optimization lever in this assignment is feature quality and data hygiene rather than model complexity.

H. Actionable Next-Step Roadmap

A practical continuation roadmap can be staged as follows:

- 1) **Cross-validation benchmarking:** report mean and variance instead of single-split metrics.
- 2) **Probability calibration:** evaluate reliability curves and Brier score for threshold-dependent deployment.
- 3) **External-signal enrichment:** add domain age, ASN reputation, DNS TTL volatility, and redirection chain depth.
- 4) **Cost-sensitive training:** optimize objective weights for asymmetric false-positive/false-negative costs.
- 5) **Monitoring pipeline:** track data drift and feature-distribution shifts over time.

This roadmap preserves the assignment’s lightweight modeling philosophy while advancing toward operational-grade phishing and spam defense.

IX. REPRODUCIBILITY APPENDIX AND ARTIFACT MANIFEST

A. Environment and Execution Protocol

All experiments and report figures were generated in the project-local virtual environment:

```
source /Users/tahamajs/Documents/uni/venv/bin/activate
cd /Users/tahamajs/Documents/uni/NLP/nlp-assignments-spring-202
pip install tldextract
jupyter nbconvert --to notebook --execute answer/NLP_HW2.ipynb
--output /tmp/ca2_exec_check.ipynb --ExecutePreprocessor.time
```

The executed notebook was then promoted back to the deliverable path so that outputs remain embedded in the submission artifact.

B. Deterministic Data Contracts

The notebook now uses fixed project paths, removing ambiguity from execution context:

- datasets/q1/emails_1.csv
- datasets/q1/emails_2.csv
- datasets/q2/urls.csv

Label normalization is explicit and validated at runtime:

- spam/ham $\rightarrow$ 1/0 for emails_1.csv
- Prediction (or status) coerced to integer for emails_2.csv
- phishing/legitimate $\rightarrow$ 1/0 for urls.csv

This design prevents silent fallback behavior and ensures reproducible class semantics across runs.

C. Figure Manifest

Table IV lists all figures that are generated automatically by notebook execution and consumed by this report.

D. Complete Feature Dictionary for Q2

For reproducibility and future extension, Table V documents all URL features used in the combined model, grouped by engineering family.

E. LaTeX Build Procedure

The IEEE report is compiled with a standard bibliography pipeline:

```
cd answer/report
pdflatex -interaction=nonstopmode main.tex
bibtex main
pdflatex -interaction=nonstopmode main.tex
pdflatex -interaction=nonstopmode main.tex
pdfinfo main.pdf
```

This generates the final PDF at:

answer/report/main.pdf

F. Validation Checklist

The final submission was validated against these criteria:

- 1) Full notebook execution completes with zero cell errors.
- 2) Real datasets are loaded from datasets/ paths without fallback demo mode.
- 3) All expected figure files exist and are non-empty.
- 4) IEEE LaTeX report compiles successfully with bibliography.
- 5) Report includes all required assignment analyses for both questions.

G. Submission-Readiness Notes

The current artifact set is designed to be grader-friendly:

- Notebook contains executable code and generated outputs.
- Report references only local generated figures.
- Paths are deterministic and independent of notebook launch location.
- Dependencies are explicit and minimal.

Together, these properties reduce ambiguity during evaluation and make the submission resilient to environment differences.

TABLE IV: Generated figure artifact manifest

Filename	Purpose	Used in section
q1_confusion_matrices.png	Side-by-side confusion matrices for from-scratch Q1 models	Q1 results
q1_roc_pr_curves_lr.png	ROC and Precision-Recall curves for Q1 Logistic Regression	Q1 results
q1_top_words_spam_ham.png	Top class-specific BoW terms from emails_1.csv	Q1 results
q1_model_comparison_bar.png	Metric bar chart for Q1 model comparison	Q1 results
q2_structural_distributions.png	Histograms of structural URL features	Q2 results
q2_correlation_heatmap_all.png	Correlation structure of combined URL features	Q2 results
q2_structural_boxplots.png	Class-conditional spread for key structural features	Q2 results
q2_pairplot_sample.png	Pairwise sampled visualization of combined features	Q2 results
q2_top_mean_abs_correlation.png	Top features by mean absolute correlation	Q2 results
q2_roc_combined_features.png	ROC comparison on combined feature set	Q2 results
q2_pr_combined_lr.png	PR curve for combined-feature Logistic Regression	Q2 results
q2_lr_feature_importance.png	Positive/negative coefficient importance plot	Q2 results

TABLE V: Complete feature dictionary used in the combined Q2 model

Feature	Family	Description and intended signal
host_len	Structural	Length of host segment; unusually long hosts can indicate obfuscation or generated domains.
path_len	Structural	Number of characters in URL path; phishing pages often include long nested paths.
url_len	Structural	Total URL length; extreme length can correlate with attack payload stuffing.
num_params	Structural	Number of query parameters; excessive parameters may indicate tracking or camouflage.
has_fragment	Structural	Binary indicator for fragment usage; sometimes used to hide payload-like tokens.
digits_in_host	Structural	Count of numeric characters in host; high counts can indicate non-human domain patterns.
length_url	Statistical	Raw URL length from content-oriented extraction; provides a second length perspective.
ratio_digits_url	Statistical	Ratio of digits to full URL length; captures aggressive numeric token usage.
longest_words_raw	Statistical	Length of longest alphabetic token after split; can expose generated token artifacts.
sub_len	Creative	Subdomain length; deeply nested or very long subdomains are common in phishing.
has_ip	Creative	Binary indicator for IP-literal host usage instead of domain names.
at_sign	Creative	Presence of '@' in URL; classic obfuscation pattern in malicious links.
dash_in_host	Creative	Presence of '-' in host; often used in brand impersonation domains.
dot_count_host	Creative	Number of dots in host; indicates domain depth and possible spoofing complexity.
slash_count_path	Creative	Count of path separators; deeper paths may correlate with deceptive navigation.
percent_encoding	Creative	Binary indicator for URL percent encoding; sometimes used to hide suspicious strings.
suspicious_words	Creative	Count of suspicious security/payment words (verify, update, login, secure, account).
has_https	Creative	HTTPS usage indicator; useful but not sufficient since phishing can also use TLS.

X. OPERATIONAL APPENDIX: END-TO-END EXECUTION RECIPE

This appendix provides an explicit command-level runbook so the complete artifact can be regenerated without ambiguity. The intent is to make grading and independent verification straightforward, even if the notebook is executed on a fresh machine with only the repository and Python environment available.

A. Command-Level Runbook

```
# 1) Activate environment and move to project root
source /Users/tahamajs/Documents/uni/venv/bin/activate
cd /Users/tahamajs/Documents/uni/NLP/nlp-assignments-spring-2023/NLP_HW2

# 2) Verify key files
ls answer/NLP_HW2.ipynb
ls datasets/q1/emails_1.csv
ls datasets/q1/emails_2.csv
ls datasets/q2/urls.csv

# 3) Ensure required Python dependencies are present
python -c "import pandas, numpy, sklearn, matplotlib, seaborn, tldextract"

# 4) Execute notebook non-interactively
MPLBACKEND=Agg jupyter nbconvert --to notebook --execute answer/NLP_HW2.ipynb \
--output /tmp/ca2_exec_check.ipynb --ExecutePreprocessor.timeout=3600

# 5) Promote executed notebook as final deliverable
cp /tmp/ca2_exec_check.ipynb answer/NLP_HW2.ipynb

# 6) Validate expected figure outputs
ls answer/report/figures/q1_confusion_matrices.png
ls answer/report/figures/q1_roc_pr_curves_lr.png
ls answer/report/figures/q1_top_words_spam_ham.png
ls answer/report/figures/q1_model_comparison_bar.png
ls answer/report/figures/q2_structural_distributions.png
ls answer/report/figures/q2_correlation_heatmap_all_features.png
ls answer/report/figures/q2_structural_boxplots.png
ls answer/report/figures/q2_pairplot_sample.png
ls answer/report/figures/q2_top_mean_abs_correlations.png
ls answer/report/figures/q2_roc_combined_features.png
ls answer/report/figures/q2_pr_combined_lr.png
ls answer/report/figures/q2_lr_feature_importance.png

# 7) Build IEEE report
cd answer/report
pdflatex -interaction=nonstopmode main.tex
bibtex main
pdflatex -interaction=nonstopmode main.tex
pdflatex -interaction=nonstopmode main.tex

# 8) Check final page count and PDF metadata
pdftinfo main.pdf | rg '^Pages|^Title'

# 9) Optional quality checks
# - Open main.pdf and visually inspect all figure captions and references
# - Confirm no missing figures in log
# - Confirm model tables match notebook outputs

# 10) Final artifact set
# - answer/NLP_HW2.ipynb
# - answer/report/main.pdf
# - answer/report/figures/*.png
# - answer/report/main.tex + section files + references.bib
```

B. Expected Observable Outcomes

A successful run should satisfy the following observable outcomes:

- 1) Notebook execution terminates without `CellExecutionError`.
- 2) Exactly twelve report figures are present and non-empty.
- 3) The report compiles with IEEEtran and includes all assignment sections.

- 4) Final PDF page count is strictly greater than ten pages.

This appendix intentionally duplicates procedural information from earlier sections in a command-first format, because reproducibility failures in assignments frequently occur in operational details rather than in modeling logic.

REFERENCES

- [1] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. Springer, 2009.
- [2] C. D. Manning, P. Raghavan, and H. Schütze, "Introduction to information retrieval," 2008.
- [3] A. McCallum and K. Nigam, "A comparison of event models for naive bayes text classification," *AAAI Workshop on Learning for Text Categorization*, 1998.
- [4] M. Chandrasekaran, K. Narayanan, and S. Upadhyaya, "Phishing email detection based on structural properties," in *New York State Cyber Security Conference*, 2006.
- [5] J. Ma, L. Saul, S. Savage, and G. Voelker, "Learning to detect malicious urls," in *ACM Transactions on Intelligent Systems and Technology*, 2011.
- [6] M. Aburrous, M. A. Hossain, K. Dahal, and F. Thabtah, "Intelligent phishing detection system for e-banking using fuzzy data mining," *Expert Systems with Applications*, vol. 37, no. 12, pp. 7913–7921, 2010.
- [7] J. Schlund and contributors, "tldextract: Accurate url domain parsing," <https://github.com/john-kurkowski/tldextract>, 2025.
- [8] F. Fawcett, "An introduction to roc analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.